

Stage 1: Ownership Screen

What was the underlying customer or business problem?

The underlying problem was that phishing emails continue to cause financial loss, operational disruption, and security incidents for both individuals and organizations. While many users are aware of phishing, identifying sophisticated phishing emails is becoming increasingly difficult.

Our goal was to build a system that could automatically detect phishing emails within Outlook while also helping educate users about why an email was considered suspicious. We wanted to reduce the likelihood of successful phishing attacks while improving user awareness and decision-making.

What assumptions were unclear at the beginning, and how did you resolve them?

At the beginning, several assumptions were unclear.

First, we did not know which machine learning approach would provide the best balance between accuracy, training time, and deployment complexity. We also did not know how much improvement additional features such as URL scanning, attachment scanning, and email metadata would provide compared to content-based classification alone.

Another challenge was understanding how Microsoft Graph APIs and Outlook integrations would fit into the overall architecture since none of us had significant experience with those technologies.

We tackled the uncertainties through research, experimentation, and prototyping. We evaluated different machine learning approaches, tested multiple datasets, built proofs of concept with Microsoft Graph, and continuously validated assumptions through testing before committing to final implementation decisions.

How did you decide scope and trade-offs?

We started by identifying the core outcome we wanted to achieve: reliable phishing detection inside the user's email workflow.

Because the project timeline was limited, we focused on the highest-value features first. These included email classification, Outlook integration, URL and attachment scanning, explanation generation, and a dashboard for reviewing results.

One major trade-off involved balancing model performance against training complexity and resource requirements. We experimented with different approaches and ultimately selected an XGBoost-based solution using SBERT embeddings because it provided strong accuracy while remaining practical to train and deploy.

We also intentionally prioritized core functionality over additional features that would increase development complexity without significantly improving the project's primary objective.

What did you test, how did you validate release readiness, and what went wrong?

For the machine learning component, we evaluated model performance using training, validation, and test datasets to ensure the classifier generalized well and met our target accuracy requirements.

For the application itself, we implemented automated tests covering key user workflows such as email scanning, classification, URL and attachment analysis, dashboard functionality, exporting results, user feedback, explanation generation, and settings management.

One challenge we encountered was the complexity of integrating multiple services and external APIs, particularly Microsoft Graph and Outlook-related functionality. We also spent significant time cleaning and preparing datasets because data quality had a major impact on model performance.

What changed after launch based on feedback or data?

As development progressed and we reviewed results, we expanded the solution beyond basic classification.

Initially, the focus was primarily on determining whether an email was phishing or safe. However, we realized that users also needed context and explanations to understand why a classification was made. From that, we introduced additional capabilities such as explanation generation, dashboard functionality, export features, and Virus Total URL and attachment analysis.

How did you use AI, if at all, and where did you have to step in with human judgment?

AI was used primarily as a development accelerator rather than a replacement for engineering judgment. I used AI-assisted tools to understand unfamiliar technologies, explore implementation approaches, generate starter code, troubleshoot issues, and accelerate development while working with Microsoft Graph, Outlook integrations, React, Go, and machine learning workflows.

However, architectural decisions, integration work, debugging, testing, and validation remained my responsibility. AI could generate suggestions, but I had to determine whether they fit the project's requirements, architecture, security considerations, and business goals. Human judgment was important deciding trade-offs, model performance, integrating system components, and ensuring the final solution worked correctly end-to-end.